Laboratório #1: Introdução a Linguagem Kotlin

Introdução

Kotlin [1] é uma linguagem moderna recomendada pela Google para desenvolvimento Android. Oferece:

- Sintaxe conscisa
- Null safety
- Recursos funcionais como lambdas e operações de colecção
- Funções de extensão
- classes de dados e tipos selados

Este guia de laboratório segue o princípio “aprende fazendo” e é desenhado para programador Java. O estudo da tecnologia Kotlin será realizado enquanto se desenvolve uma aplicação simples console. A especificação da aplicação de amostra está no apêndice A1.

1. Objectivo

Dominar os fundamentos da linguagem Kotlin, especificamente:

- Variáveis (val, var, $, tipos, ?), Null safety
- Colecções (List, Set e Map)
- Funções e lambdas
- Controlo de fluxo (if, if-else, when, for, while e do-while)
- Orientação a objectos (class, data class, abstract class, interface, herança, polimorfismo, etc) e pacotes.

2. Fundamentos da Programação Procedimental com Kotlin

Instale o JDK 21 e o Intellij. Crie o projecto com o nome Hello (precisa ter acesso a internet para que possa descarregar os pacotes durante a criação do projecto):

João Costa / joaojdacosta@gmail.com / https://github.com/joaojdacosta , (2026, Outubro)

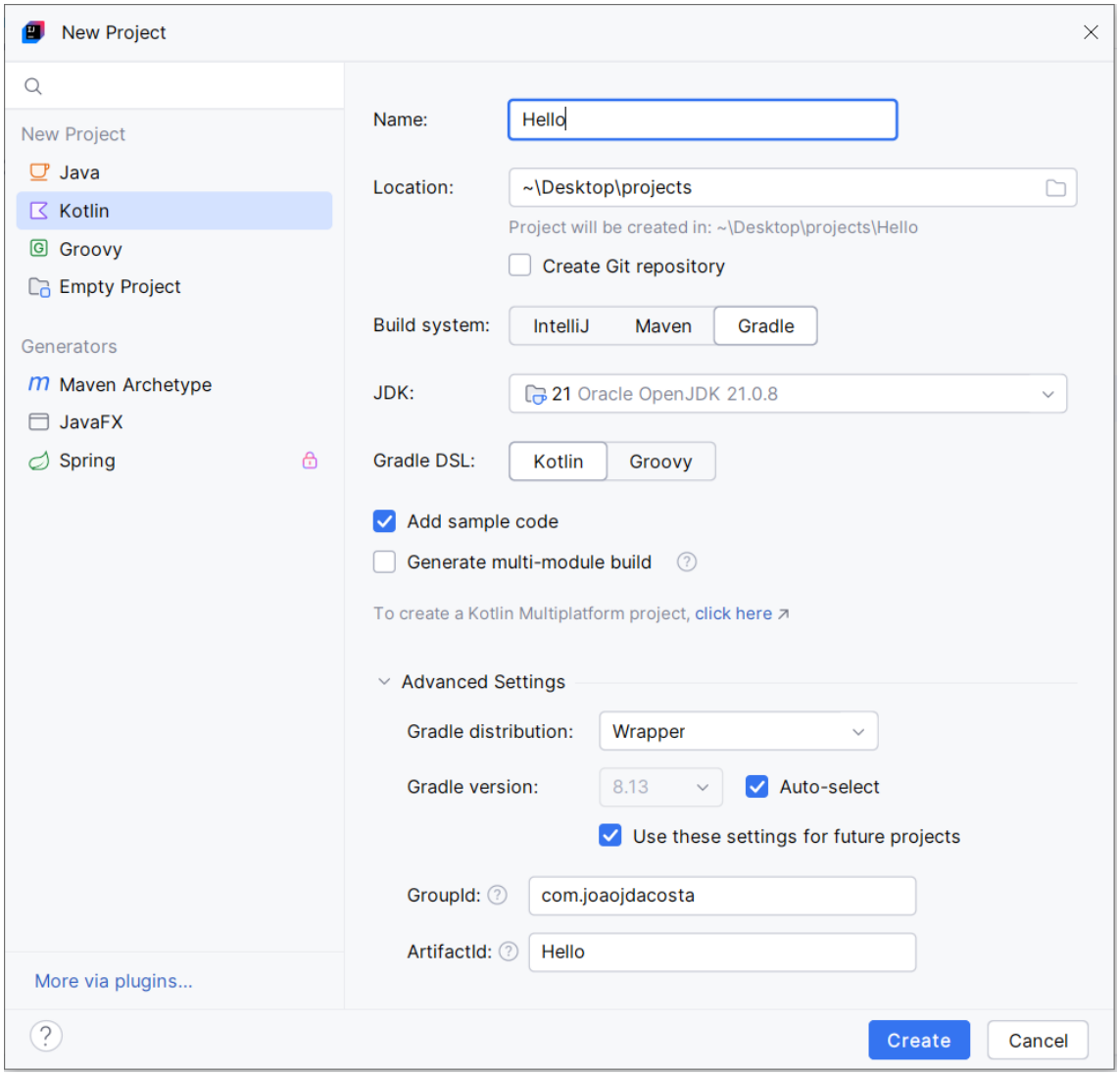
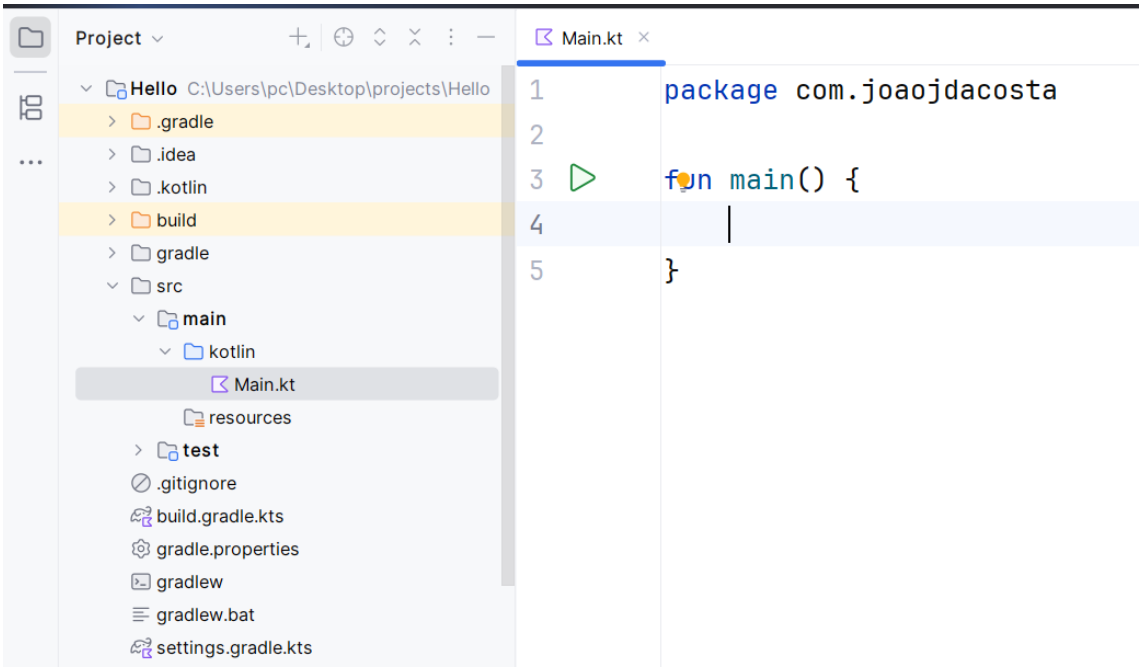


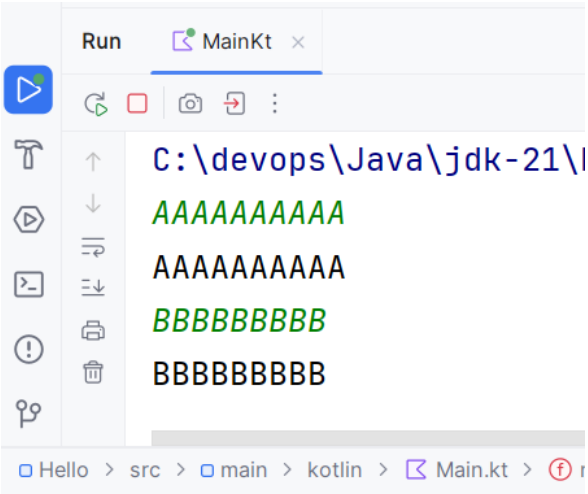
Figura 1: Tela Novo Projecto.

Em novo projecto selecciona **Kotlin**, nome do projecto **Hello**, defina uma **local** para guardar os seus projectos, não precisa criar um repositório Git, para o sistema de construção (build system) selecciona **Gradle**, repare que já reconhece o JDK, Gradle DSL (Domain-Specific Language) , em configurações avançadas (Advanced Settings) adicione o GroupId que o pacote base do projecto (similar o que é feito no projecto java) e clique em criar (create).

Esta acção cria o projecto com o ficheiro Main.kt contendo a função main e o pacote base (https://kotlinlang.org/docs/packages.html#imports) criado:

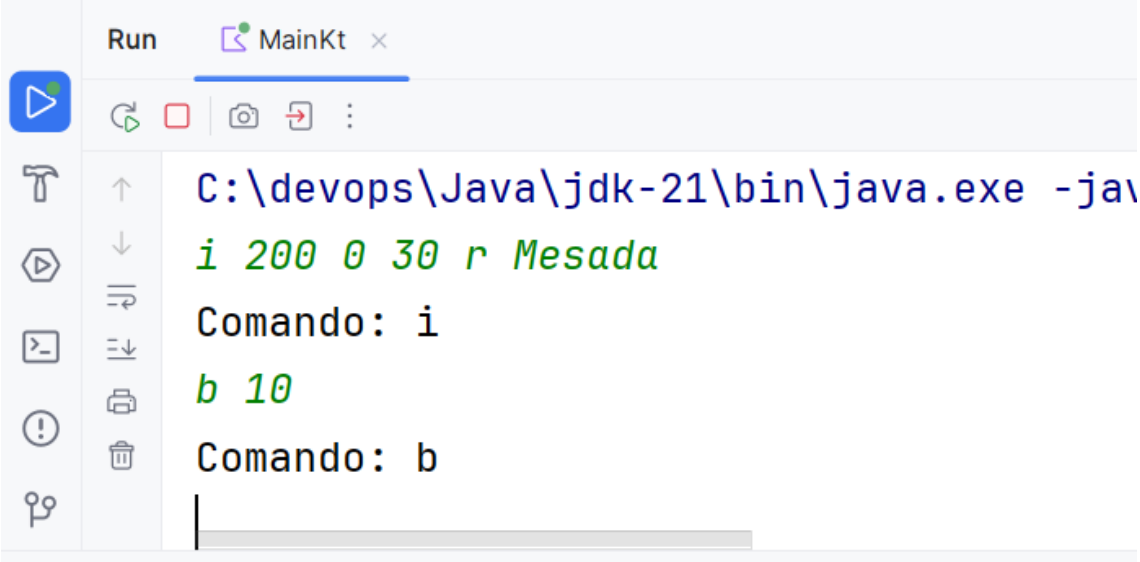


Declare a variável cmd do tipo String (https://kotlinlang.org/docs/basic-syntax.html#variables e https://kotlinlang.org/docs/types-overview.html#basic-types) e a seguir crie um ciclo (https://kotlinlang.org/docs/control-flow.html#statements) para repetir infinitamente os seguintes (1) ler uma string e guardar na variável cmd (**readln**) e (2) apresentar o conteudo da variável (**println**).



Agora implementa a lógica para identificar o comando e chamar a função responsável (cada comando uma função que recebe uma string e retorna o tipo a seu gosto, https://kotlinlang.org/docs/functions.html).

João Costa / joaojdacosta@gmail.com / https://github.com/joaojdacosta , (2026, Outubro)



Agora defina as estruturas de dados para armazenar um conjunto de movimentos, cada movimento é composto pelos seguintes atributos: montante, data, periodicidade, tipo e descrição).



O exemplo acima é uma estrutura de dados do tipo lista, que armazena objectos do tipo Movimento. O movimento é o modelo composto pelos atributos listados acima.

No entanto, pode seleccionar a estrutura de dados mais adequado para o problema conforme o seu domínio. Link interessante:

- https://kotlinlang.org/docs/collections-overview.html
- https://kotlinlang.org/api/core/kotlin-stdlib/kotlin.collections/
- https://developer.android.com/codelabs/basic-android-kotlin-collections#0

A seguir faça a implementação das funções, sugerimos a seguinte sequência e encorajamos os testes unitários antes de prosseguir para função seguinte. Sugestão da ordem de implementação:

- Implementar o comando inserir movimento, ou um conjunto de movimentos.
- Implementar o comando para mostrar todos os movimentos ocorridos numa determinada data.
- Implementar o comando para remover um determinado movimento.

- Implementar o comando para calcular o balanco numa determinada data, incluindo todos os movimento ocorridos nessa data.
- Implementar o comando para alterar o valor mínimo do balanço que não invalida um orçamento, cujo valor por defeito é 0.
- Implementar o comando para verificar a validade do orçamento.
- Implementar o comando para determinar a primeira data em que uma determinada despesa se torna comportável, sem invalidar o orçamento.

Nota importante:

- Precisa garantir a validação de entrada e obedecer a especificação.
- Atenção as boas práticas: https://kotlin.dev.br/tutoriais/boas-praticas-kotlin/ e https://kotlinlang.org/docs/coding-conventions.html

3. Fundamentos da Programação Orientada a Objecto com Kotlin

No ponto anterior, o programa é procedimento seguindo uma estrutura similar a seguinte:

```
main()

inserir()

lista()

remover()

balanco()

setValorMinimo()

verificar()

comportavel()
```

Agora dado o seu conhecimento do paradigma orientação a objecto e padrões de projecto deve refactorar o seu código para estruturar em: pacotes, classes, etc.

Extende a solução para vários utilizadores (identificado com um `email` e `password`). Cada utilizador tem um conjunto de movimentos e pode realizar as operações (inserir, listar, remover, etc) após inicio de sessão. O início de sessão obedece o seguinte comando:

```
u email password
```

onde `u` é o comando para início de sessão e `email` e `password` parâmetros do comando `u`. O comando verifica se o utilizador existe, através da correspondência do `email` e a `password`. Se existe, então ficam disponíveis os comandos (`i`, `l`, `r`, `b`, `m`, `v`,`w`) no contexto do utilizador autenticado (i.e., todas as operações é sobre os movimentos realizados pelo utilizador autenticado). Caso não exista, o sistema cria um novo utilizador com as credenciais (`email` e `password`) e inicia sessão onde ficam disponíveis os comandos do implementados acima e, obvimente, sem movimento por se tratar de um novo utilizador. O `email` deve ser um atributo chave (sem duplicação). Os comandos (`i`, `l`, `r`, `b`, `m`, `v`,`w`) ficam disponível após início de sessão. Uma sessão tem os seguintes atributos: `token` (um hash único) e `datahora`. Um utilizador tem zero ou várias sessão registadas no sistema.

O comando `exit` permite encerrar uma sessão, mas não encerra o programa.

Referências

- Kotlin, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/home.html
- Hello world, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-hello-world.html
- Tipos básicos, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-basic-types.html
- Colecções, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-collections.html
- Controlo de fluxo, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-control-flow.html
- Funções, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-functions.html
- Classes, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-classes.html
- Null safety, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-null-safety.html
- Extensão de funções, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-intermediate-extension-functions.html
- Extensão de funções, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-intermediate-extension-functions.html
- Escopo de funções, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-intermediate-scope-functions.html
- Classes, Interfaces e Objecto, (2026, 06 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-intermediate-classes-interfaces.html; https://kotlinlang.org/docs/kotlin-tour-intermediate-objects.html; https://kotlinlang.org/docs/kotlin-tour-intermediate-properties.html .

APÊNDICE

A1 – Exercício de amostra

Sistema de Orçamento Pessoal

Introdução

Neste trabalho pretende-se implementar um sistema de orçamento pessoal. O objectivo de um orçamento é controlar a forma como é utilizado o dinheiro que temos à nossa disposição.

O sistema deverá dar suporte a este objectivo. Neste caso é relativo a uma pessoa ou a um pequeno agregado familiar, por oposição a uma empresa que pode ter um orçamento bastante mais complicado.

Para atingir este objectivo o sistema deverá começar por registar os movimentos de dinheiro.

Para cada movimento, para além do montante envolvido, deverá ser também fornecida uma pequena descrição, bem como a data em que ocorreu. Para simplificar, vamos assumir que cada data é um número entre 0 e 999. Cada movimento pode ser uma receita ou uma despesa, o que significa que o dinheiro é recebido ou gasto, respectivamente.

O utilizador do sistema irá introduzir, através da linha de comandos, os dados relativos a cada um dos movimentos: montante, data, periodicidade, tipo (receita/despesa) e descrição.

Para facilitar esta tarefa vai ser possível inserir movimentos recorrentes, ou seja movimentos que se repetem com uma determinada periodicidade. Por exemplo uma "Mesada" pode occorrer de 30 em 30 dias. Note que esta funcionalidade é principalmente uma caracteristica da interface, pelo que internamente não é necessário guardar a informação se um movimento é periódico ou não. Desta forma podemos escrever apenas 1 comando, em vez de 1000, no caso e de um movimentodeperiodicidade 1. No entanto o estado interno do sistema seria o mesmo em ambos os casos. Deverá ser igualmente possível remover movimentos do orçamento.

Concluída a tarefa de inserir e remover movimentos, o utilizador poderá utilizar o sistema para calcular, em cada data, qual é o balanço entre as despesas e as receitas. Sendo que é espectável, para uma boa gestão, que este valor seja sempre maior ou igual a 0. De facto é desejável que seja sempre maior ou igual do queumdadovalormínimo, definidopeloutilizador.

Caso esta condição não seja verificada o sistema deverá informar o utilizador que o orçamento é inválido.

Osistema deverá ainda auxiliar o utilizador a produzir orçamentos válidos. Um orçamento pode tornar-se inválido quando o utilizador introduz uma determinada despesa. Poderá até verificar-se que na data da despesa o balanço permaneça maior do que o mínimo permitido, no entanto essa despesa poderá invalidar o orçamento numa despesa futura. Considere o seguinte exemplo:

Tabela 1: Exemplo do comportamento do sistema.

Montante	Data	Tipo	Descrição	Balanço
100	0	Receita	Mesada	100
50	15	Despesa	Livro	50
100	30	Receita	Mesada	150

João Costa / joaojdacosta@gmail.com / https://github.com/joaojdacosta , (2026, Outubro)

120	40	Despesa	PC	30
100	60	Receita	Mesada	130

Para um valor mínimo de balanço de 30 este orçamento é válido. Caso o valor mínimo seja maior do que 30, o orçamento é inválido. Consideremos agora que o valor mínimo é 20 e que inserimos uma despesa de 20 na data 35. Nessa data, imediatamente após a despesa, o balanço será de 130. No entanto, na data 40 a despesa com o "PC" baixa o balanço para 10, que é menor do que 20 e portanto o orçamento resultante é inválido.

Para evitar este tipo de situação o sistema suporta uma operação de consulta tal que, dada uma data e um montante, devolve a primeira data após a data indicada em que é possível introduzir uma despesa com o montante indicado, sem invalidar o orçamento. Neste caso para a data 35 e o valor 20 a resposta deve ser 60. Pode acontecer que a despesa nunca seja possível, como seria o caso se a última mesada não existisse.

Especificação do programa

O programa deverá receber como dados de entrada uma sequência de comandos dos seguintes

tipos:

- inserção/remoção de movimentos;
- consulta de balanço;
- verificação da validade do orçamento;
- consulta de data de despesa possível.

O programa deverá funcionar de forma interactiva, respondendo a cada comando do utilizador.

Dados de Entrada

O programa deverá ler os dados de entrada a partir da console. Cada linha corresponde a um comando, cujo tipo é identificado pela letra colocada no início da linha. Os argumentos do comando são dados na mesma linha e aparecem separados por um espaço em branco. A especificação dos comandos é a seguinte:

1. O comando para inserir um movimento, ou um conjunto de movimentos, tem o seguinte formato:
    - aletra 'i';
    - o valor (montante) do movimento;
    - um número com a data do primeiro movimento;
    - um número com a periodicidade entre movimentos; caso este valor seja o o movimento é único (não periódico);
    - o caracter 'r' caso o movimento seja uma receita, ou o caracter 'd' caso seja uma despesa;
    - uma sequência de no máximo 60 caracteres, não brancos, com a descrição do movimento.

João Costa / joaojdacosta@gmail.com / https://github.com/joaojdacosta , (2026, Outubro)

2. O comando para mostrar todos os movimentos ocorridos numa determinada data tem o seguinte formato:
    - aletra 'l';
    - um número com a data para qual se pretende visualizar os movimentos.
3. O comando para remover um determinado movimento tem o seguinte formato:
    - aletra 'r';
    - a data em que o movimento ocorreu;
    - o índice do movimento, dado pelo comando anterior.
    - Note que este comando apenas retira 1 movimento, mesmo que o respectivo comando de inserção tenha inserido vários.
4. O comando para calcular o balanco numa determinada data, incluindo todos os movimento ocorridos nessa data, tem o seguinte formato:
    - aletra 'b';
    - a data para a qual se pretende obter o balanço.
5. O comando para alterar o valor mínimo do balanço que não invalida um orçamento, cujo valor por defeito é 0, tem o seguinte formato:
    - aletra 'm';
    - o novo valor mínimo.
6. O comando para verificar a validade do orçamento tem o seguinte formato:
    - aletra 'v'.
7. O comando para determinar a primeira data em que uma determinada despesa se torna comportável, sem invalidar o orçamento, tem o seguinte formato:
    - aletra 'w';
    - a primeira data a considerar;
    - o valor da despesa.

Assuma que os dados de entrada respeitam sempre a especificação apresentada, não sendo portanto necessário implementar mecanismos de verificação de situações de erro. Considere ainda as seguintes restrições:

- todas as datas são números inteiros entre 0 e 999, sendo a última data do orçamento 999.
- em cada data poderão existir no máximo 100 movimentos;
- a descrição de um movimento não contém mais de 60 caracteres.

Dados de Saída

Após processar cada um dos comandos indicados nos dados de entrada, o programa deverá escrever no na console o respectivo resultado, caso exista. Os únicos comandos que produzem resultados são: 'l', 'b', 'v' e 'w'. Os restantes comandos são silenciosos.
O comando 'l', para cada movimento ocorrido na data indicada, apresenta numa linha os seguintes campos separados por um espaço:

- um número inteiro, indicando a posição relativa do movimento (índice), cujo valor começa em 0 e é incrementado em 1 por cada movimento;
- o montante associado ao movimento em questão;
- o caracter 'r' ou 'd', conforme o tipo de movimento seja uma receita ou uma despesa, respectivamente;
- a descrição do movimento em causa.

Os movimentos devem ser apresentados pela ordem pela qual foram inseridos. Quando um movimento é retirado do conjunto de movimentos de uma determinada data, a ordem relativa dos restantes movimentos deverá permanecer inalterada. Contudo os índices de posição devem ser actualizados.

O comando 'b' deverá apresentar uma linha contendo apenas o valor do balanço na data indicada.

O comando 'v' deverá apresentar uma linha contendo a mensagem Valido ou a mensagem Invalido, conforme o orçamento seja válido ou inválido, ou seja se cumpre ou não o limite mínimo de balanço definido.

O comando 'w' deverá apresentar uma linha contendo apenas a primeira data (a partir da data indicada) em que a despesa indicada pode ser suportada, sem tornar o orçamento inválido.

Caso a despesa não seja possível em nenhuma data entre a data indicada e 999, então a linha deve conter apenas a mensagem Incomportavel.

Exemplo

Dados de Entrada

Considere a seguinte entrada (sequência de comandos):

```
i 100 0 30 r Mesada
i 50 15 0 d Livros
i 120 40 0 d PC
m 50
b 30
b 40
b 60
v
m 30
b 30
b 40
b 60
v
i 20 35 0 d Mochila
b 30
b 40
b 60
v
i 5 35 0 d Estojo
```


João Costa / joaojdacosta@gmail.com / https://github.com/joaojdacosta , (2026, Outubro)

```
i 5 35 0 d Caneta
b 60
v
l 35
r 35 0
l 35
r 35 1
l 35
r 35 0
l 35
b 60
v
b 35
w 35 20
b 999
w 35 3200
w 35 10000
```

Dados de Saída

Para os dados de entrada e na sequência de execução, teremos a seguinte saída:

```
150
30
130
Invalido
150
30
130
Valido
150
10
110
Invalido
100
Invalido
0 20 d Mochila
1 5 d Estojo
2 5 d Caneta
0 5 d Estojo
1 5 d Caneta
0 5 d Estojo
130
Valido
150
60
3230
990
Incomportavel
```


João Costa / joaojdacosta@gmail.com / https://github.com/joaojdacosta , (2026, Outubro)